

《OpenCV + Python: 图像与视频处理基础》

授课教师：王洪松

邮箱：hongsongsui@gmail.com

动画与视频

- imageio: 处理图像的接口, 包括动画图像

```
import imageio.v2 as imageio
import glob
filenames = glob.glob('*.png' ) # 静态图片存放路径
save_name_gif = "generate.gif" # 转化的GIF图片名称
# fps 就是图片切换的频率, 越大越快。
fps = 1
# 播放次数, 0表示循环播放, 1表示播放1次, 2表示播放2次, 以此类推
loop = 0
# 存放图片的列表
pics_list = [imageio.imread(image_name) for image_name in filenames]
# 生成gif
imageio.mimsave(save_name_gif, pics_list, 'GIF', fps=fps, loop=loop)
```

视频处理：FFMPEG

<https://hongsong-wang.github.io/>

- FFMPEG：专门用于处理音视频的开源库
- 下载编译好的FFMPEG可执行文件

<https://github.com/BtbN/FFmpeg-Builds/releases>

选择对应的电脑系统版本，对于windows系统，可以选

 ffmpeg-master-latest-linux64-gpl-shared.tar.xz	41.9 MB
 ffmpeg-master-latest-linux64-gpl.tar.xz	96.2 MB
 ffmpeg-master-latest-linux64-lgpl-shared.tar.xz	37 MB
 ffmpeg-master-latest-linux64-lgpl.tar.xz	86.9 MB
 ffmpeg-master-latest-linuxarm64-gpl-shared.tar.xz	34.2 MB
 ffmpeg-master-latest-linuxarm64-gpl.tar.xz	77.7 MB
 ffmpeg-master-latest-linuxarm64-lgpl-shared.tar.xz	30.9 MB
 ffmpeg-master-latest-linuxarm64-lgpl.tar.xz	71.4 MB
 ffmpeg-master-latest-win64-gpl-shared.zip	51 MB
 ffmpeg-master-latest-win64-gpl.zip	128 MB

将 “ffmpeg-master-latest-win64-gpl-shared\bin” 添加到电脑环境目录中（或把以下三个拷贝到程序运行目录）。

三个可执行文件：

ffmpeg、ffprobe、ffplay

- 转化格式
 - 抽取画面中的音频
 - 音频+视频合成
 - 视频分离成图片
`ffmpeg -i video.mp4 -r 1 -f image2 out_%04d.jpg`
(-r 帧率每秒钟转化1张)
 - 图片合成视频
`ffmpeg -ss 15 -t 25 -i video.mp4 -c:v libx264 -c:a aac`
 - 截取视频/音频
`-strict experimental clip.mp4`
 - 图片生成gif动图
`ffmpeg -i input_image_%03d.png -r 5 output_test.gif`
- ss 15 : 从第15秒开始剪切视频。
-t 25 : 指定剪切后的视频时长为25秒。
-c:v libx264: 指定视频的编码格式为libx264格式。
-c:a aac : 指定音频的编码格式为aac格式。
-strict experimental : 安全处理。

生成视频文件

<https://hongsong-wang.github.io/>

```
img1=cv2.imread('BaboonRGB.bmp')
img2=cv2.imread('LenaRGB.bmp')
rows1, cols1 = img1.shape[:2]
rows2, cols2 = img2.shape[:2]
print(img1.shape[:2], img2.shape[:2]))
# 根据小图像的大小, 在大图像上创建感兴趣区域
roi = img1[0:rows2, 0:cols2]
fourcc = cv2.VideoWriter_fourcc(*'XVID' )
out = cv2.VideoWriter('result.avi', fourcc, 80,
(cols2, rows2), True) # (width, height)
for a in range(0,1000,1):
    b=a/1000
    dst = cv2.addWeighted(roi, b, img2, 1-b, 0)
    out.write(dst)
out.release()
```



生成动图文件

<https://hongsong-wang.github.io/>

```
import cv2
import imageio
```

```
img1 = cv2.imread('BaboonRGB.bmp')
img2 = cv2.imread('LenaRGB.bmp')
rows2, cols2 = img2.shape[:2]
roi = img1[0:rows2, 0:cols2]
```

```
duration = 5 # 动画长度为5秒
```

```
fps = 5 # 每秒5帧
```

```
images = []
```

```
for i in range(duration*fps):
```

```
    b = i/(duration*fps - 1)
```

```
    dst = cv2.addWeighted(roi, b, img2, 1 - b, 0)
```

```
    b, g, r = cv2.split(dst)
```

```
    images.append(cv2.merge((r, g, b)) )
```

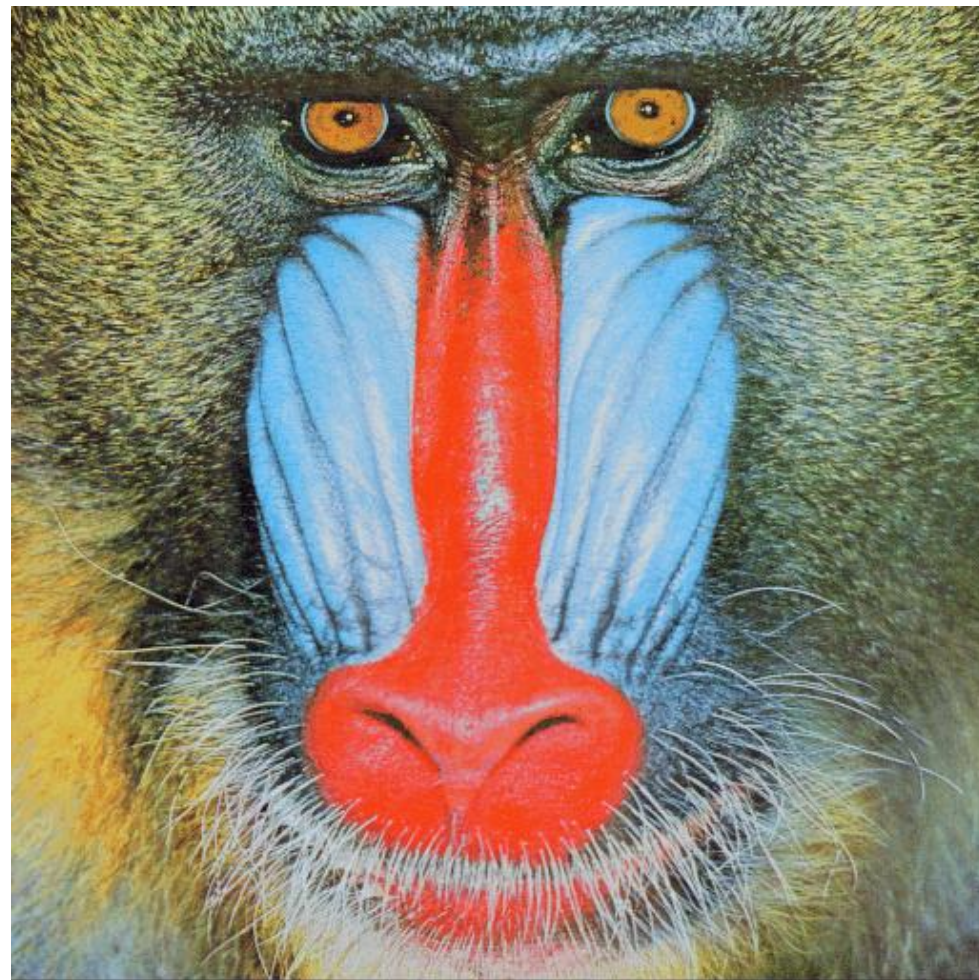
```
imageio.mimsave( "res.gif" ,images, fps=fps, loop=1) # 播放1次, loop=0循环播放
```



转场动画：平移（仿射变换）

<https://hongsong-wang.github.io/>

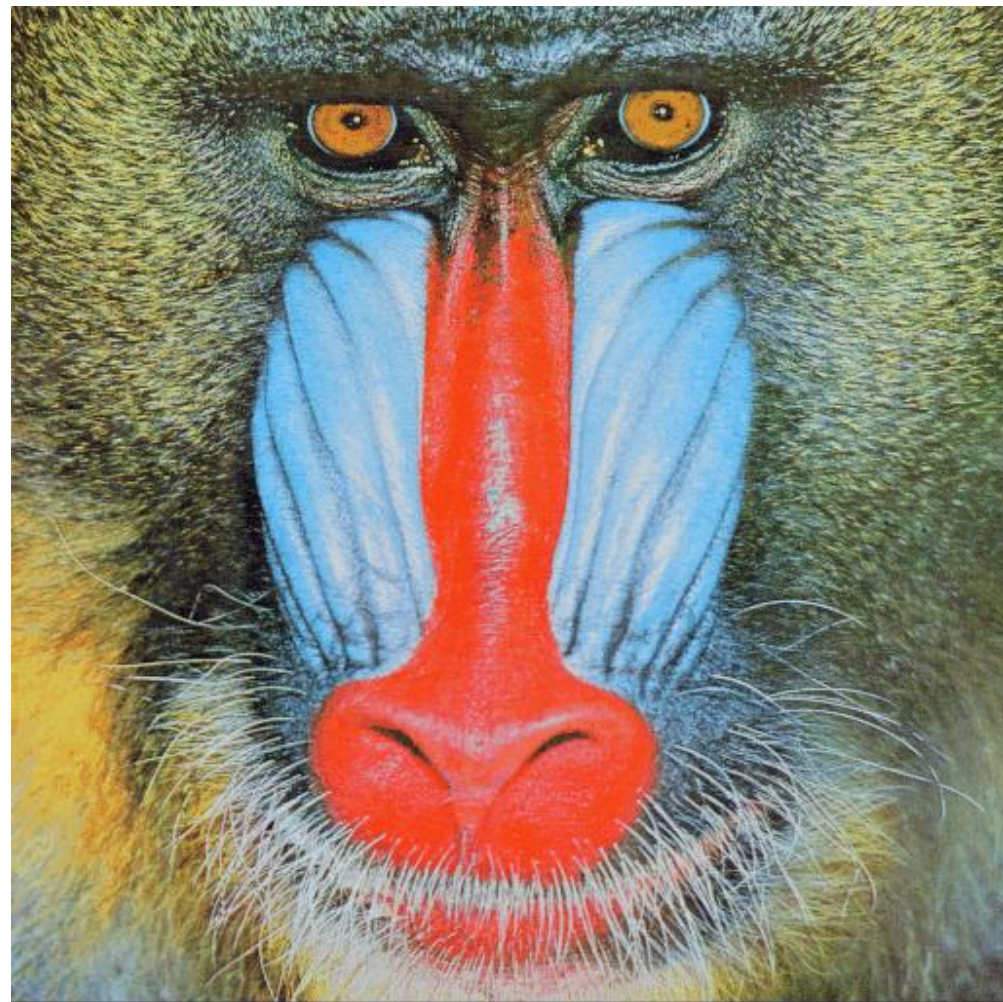
```
import cv2
import numpy as np
import imageio
img1 = cv2.imread('BaboonRGB.bmp')
img2 = cv2.imread('LenaRGB.bmp')
img = np.hstack([img1, img2])
rows, cols = img1.shape[:2]
duration = 2 # 动画长度为5秒
fps = 5 # 每秒5帧
images = []
for i in range(duration*fps):
    percent = i/(duration*fps - 1)
    x = int(percent * cols)
    M = np.float32([[1, 0, -x], [0, 1, 0]])
    dst = cv2.warpAffine(img, M, (rows, cols))
    b, g, r = cv2.split(dst)
    images.append(cv2.merge((r, g, b)) )
imageio.mimsave("res.gif", images, fps=fps, loop=1)
```



转场动画：控制变化速率

<https://hongsong-wang.github.io/>

```
import cv2
import numpy as np
import imageio
img1 = cv2.imread('BaboonRGB.bmp')
img2 = cv2.imread('LenaRGB.bmp')
img = np.hstack([img1, img2])
rows, cols = img1.shape[:2]
duration = 2 # 动画长度为5秒
fps = 5
images = []
for i in range(duration*fps):
    n = 2
    percent = (i/(duration*fps - 1))**n
    x = int(percent * cols)
    M = np.float32([[1, 0, -x], [0, 1, 0]])
    dst = cv2.warpAffine(img, M, (rows, cols))
    b, g, r = cv2.split(dst)
    images.append(cv2.merge((r, g, b)) )
imageio.mimsave("res.gif", images, fps=fps, loop=1)
```



视频关键帧

视频关键帧定义

<https://hongsong-wang.github.io/>

- 关键帧反映一个镜头的主要内容，关键帧对视频具有概括力，可用于基于内容的视频检索与分析。



光流

- 光流 (optical flow) 是空间运动物体在观察成像平面上的像素运动的瞬时速度。
- 光流是一个 2D 向量场, 可以用来显示一个点从第一帧图像到第二帧图像之间的移动。
- 光流法基本假设条件: 在连续的两帧图像之间 (目标对象的) 像素的灰度值不改变; 相邻的像素具有相同的运动。

$$I(x, y, t) = I(x + dx, y + dy, t + dt)$$

$$f_x u + f_y v + f_t = 0$$

$$f_x = \frac{\partial f}{\partial x}; \quad f_y = \frac{\partial f}{\partial y} \quad u = \frac{dx}{dt}; \quad v = \frac{dy}{dt}$$

- 邻域内的所有点都有相似的运动。Lucas_Kanade 法就是利用一个 3x3 邻域中的 9 个点具有相同运动的这一点。
- 找到这 9 个点的光流方程，用它们组成一个具有两个未知数 9 个等式的方程组，这是一个约束条件过多的方程组。一个好的解决方法就是使用最小二乘拟合。

$$\begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} \sum_i f_{x_i}^2 & \sum_i f_{x_i} f_{y_i} \\ \sum_i f_{x_i} f_{y_i} & \sum_i f_{y_i}^2 \end{bmatrix}^{-1} \begin{bmatrix} -\sum_i f_{x_i} f_{t_i} \\ -\sum_i f_{y_i} f_{t_i} \end{bmatrix}$$

```
# calculate optical flow 能够获取点的新位置  
p1, st, err = cv2.calcOpticalFlowPyrLK(old_gray,  
frame_gray, p0, None, **lk_params)
```

- Lucas-Kanade 法是计算一些特征点的光流，OpenCV 还提供了一种计算稠密光流的方法。它会图像中的所有点的光流。

```
#cv2.calcOpticalFlowFarneback(prev, next, pyr_scale, levels, winsize, iterations, poly_n,  
#poly_sigma, flags[])  
#pyr_scale – parameter, specifying the image scale (<1) to build pyramids for each image;  
#pyr_scale=0.5 means a classical pyramid, where each next layer is twice smaller than the  
#previous one.  
#poly_n – size of the pixel neighborhood used to find polynomial expansion in each pixel;  
#typically poly_n =5 or 7.  
#poly_sigma – standard deviation of the Gaussian that is used to smooth derivatives used  
#as a basis for the polynomial expansion; for poly_n=5, you can set poly_sigma=1.1, for  
#poly_n=7, a good value would be poly_sigma=1.5.  
#flag 可选 0 或 1,0 计算快, 1 慢但准确  
flow = cv2.calcOpticalFlowFarneback(prvs,next, None, 0.5, 3, 15, 3, 5, 1.2, 0)
```

- 实现滚动字幕（滚动文字动画）
 - 输入一张图像和一段文字作为该图像的字幕。
 - 把文字插入图像中，并在图像的下侧从左向右滚动，一直到最后的一行文字出现并消失。
- 基于单张图像制作动画图像
 - 输入包含前景目标和背景的一张真实彩色图像
 - 制作前景目标在背景中移动的动画图像
 - 制作前景目标局部运动的动画图像，比如人挥手的动作

- 计算稠密光流并可视化
 - 输入一个视频，分别用灰度值，RGB每个通道的数值计算光流，HSV每个通道计算光流，可视化并比较结果
- 基于关键帧插值中间帧
 - 给定一个视频，人工标记关键帧
 - 写一个程序基于关键帧自动插入中间帧